Architecture Justification: Hostel Maintenance & Issue Ticketing System

Name: Sonia Sharma **SRN:** PES1UG24AM904
Section: C **Team No.:** 7

Architecture Selection: We chose Layered Architecture for the Hostel Maintenance & Issue Ticketing System.

Architectural Choice: We selected a Layered Architecture with eight components arranged in three layers. Each layer communicates only with the layer directly below it, through the provided and required interfaces listed below.

Layer	Components	Interfaces
Presentation	Resident Portal; Warden & Staff Console	Requires ITicketSubmission, IIntakeDashboard, IDispatchControl, IStatusUpdate, ISignOffClosure
Business	Ticket Intake & Triage Service (FR-001, FR-005); Staff Dispatch Service (FR-002); Job Status Tracking Service (FR-003); SLA & Closure Service (FR-004, NFR-001)	Provides the five interfaces above; connected internally by INewTicketFeed, IWorkOrder and IStatusEvent; requires IDataAccess
Data	Data Access & Media Storage Component; Relational Database & Media Store	Provides IDataAccess; connected internally by IStorage

Reason 1: The workflow splits cleanly into stages. A ticket moves in one direction: a resident raises it (room number, category such as plumbing or electrical, and an optional photo), the warden triages it and assigns a technician, the technician updates progress, and the job is signed off and closed. Each stage is its own Business layer service and passes work forward through a narrow interface (INewTicketFeed, then IWorkOrder, then IStatusEvent). A change such as adding a new issue category or changing how priority is calculated therefore stays inside Ticket Intake & Triage, while dispatch, status tracking, closure and the database schema remain unchanged.

Reason 2: The right amount of structure for a campus-scale system. The system serves one hostel or campus intranet with a few hundred users and is built by a small student team. Layers let each team member build and unit test one service against interface mocks in parallel. Microservices would add separate deployments, network calls between services and distributed data consistency, none of which this workload needs. Plain Client-Server would let UI code query the database directly, mixing presentation and data logic.

Security Advantage: The Data Access & Media Storage Component is the only way to reach persistent data, and the portals cannot open a database connection. Every read and write goes through IDataAccess, which checks the caller's role (a resident cannot change a work order or close a ticket), validates input, and encrypts resident identity data and uploaded photos at rest using AES-256. Because all security rules live in one place, they are applied consistently and are easy to audit.

Performance Benefit: Business services keep no per-user session state, so additional copies can run behind a load balancer when ticket volume peaks, for example in the first weeks of a semester. The Ticket Intake & Triage Service also keeps the list of open, unassigned tickets in memory, so the warden's dashboard (FR-005) loads in under 2 seconds without repeatedly scanning the tickets table.

Trade-off: Passing every request through each layer adds a small cost per call, and the services cannot be deployed or scaled fully independently as microservices can. We accept this in exchange for simpler development, clear ownership of each component, and data security handled in one place.